

Документация API сетевого сервера игры

1. Назначение

Сервер реализует TCP-протокол обмена JSON-сообщениями между игровым клиентом и Unity-сценой. По этому API клиент:

1. подключается к серверу и закрепляется за командой;
2. получает состояние мира в реальном времени;
3. отправляет команды агентам;
4. получает события начала/окончания игры и результат матча.

2. Запуск

Для запуска нужно сначала запустить сервер с игрой, далее запустить реализованных клиентов с выбором команд, после чего появится кнопка «Начать игру», после этого начнётся игра, где можно будет передавать действия серверу.

3. Транспорт и общие правила

3.1. Протокол

Транспорт: TCP

Порт по умолчанию: 8080

Формат сообщений: JSON

Разделитель сообщений: перевод строки \n

Каждое сообщение должно быть отправлено как одна JSON-строка, завершённая \n.

Пример:

```
{"team": "blue"}
```

Кодировка: UTF-8

3.2. Модель обмена

Сервер работает в режиме:

1. Клиент подключается по TCP.

2. Клиент отправляет сообщение выбора команды.
3. Сервер отвечает подтверждением или отказом.
 - 3.1.1. После подтверждения клиент получает события игры и периодические снимки состояния мира.
5. Клиент отправляет команды в формате batch-сообщений.

4. Подключение к серверу

4.1. Адрес

```
HOST = 127.0.0.1  
PORT = 8080
```

4.2. Первое сообщение от клиента

Сразу после подключения клиент должен отправить JSON с командой выбора команды:

```
{  
  "team": "blue"  
}
```

или

```
{  
  "team": "red"  
}
```

4.3. Правила выбора команды

Одна команда может быть занята только одним активным клиентом.

Если команда уже занята, сервер отклонит подключение.

После отключения клиента команда освобождается.

4.4. Ответ сервера на вход

Успешное подключение

```
{  
  "type": "joinAccepted",  
  "playerId": "player_1",  
  "team": "blue"  
}
```

Поля:

type - всегда joinAccepted

playerId - внутренний идентификатор игрока, назначаемый сервером

team - команда клиента

Отказ в подключении

```
{
  "type": "joinRejected",
  "reason": "occupied"
}
```

Поля:

type - всегда joinRejected

reason - причина отказа

Сейчас сервер использует причину:

occupied - команда уже занята

5. Состояние игры

Сервер рассылает клиентам снимки состояния мира. Эти сообщения приходят регулярно с интервалом, заданным параметром:

`_updateInterval = 0.1f`

То есть примерно 10 обновлений в секунду.

5.1. Структура сообщения состояния мира

```
{
  "tick": 123,
  "gameTime": 45.67,
  "gameState": "InGame",
  "bases": [],
  "agents": [],
  "treasures": [],
  "golems": []
}
```

5.2. Поля верхнего уровня

tick - Целое число. Текущий серверный тик.

gameTime - Число с плавающей точкой. Значение Time.time из Unity.

gameState - Строка состояния игры. (Lobby, InGame, Finished)

6. Форматы сущностей мира

6.1. BaseDto - база команды

```
{
  "team": "blue",
  "pos": { "x": 0, "y": 0, "z": 0 },
  "points": 12
}
```

Поля:

team - команда базы (blue / red)

pos - позиция базы

points - текущее количество очков

6.2. AgentDto - агент

```
{
  "agentId": "-12385",
  "team": "blue",
  "pos": { "x": 10, "y": 0, "z": 5 },
  "hasTreasure": false,
  "isStunned": false
}
```

Поля:

agentId - уникальный идентификатор агента

team - команда агента

pos - позиция агента

hasTreasure - несёт ли агент сокровище

isStunned - оглушён ли агент

6.3. TreasureDto - сокровище

```
{
  "id": "4521",
  "pos": { "x": 3, "y": 0, "z": 9 },
  "isPicked": false
}
```

Поля:

id - уникальный идентификатор объекта

pos - позиция сокровища

isPicked - подобрано ли сокровище

5.4. GolemDto - голем

```
{
  "id": "781",
  "pos": { "x": 7, "y": 0, "z": -2 },
  "currentSpeed": 3.5,
  "state": "Patrol"
}
```

Поля:

id - уникальный идентификатор голема

pos - позиция голема

currentSpeed - текущая скорость

state - текущее состояние AI

5.5. Vector3Dto

```
{  
  "x": 10,  
  "y": 0,  
  "z": 5  
}
```

7. Команды клиента

Клиент отправляет команды в виде batch-сообщения:

```
{  
  "actions": [  
    {  
      "Id": "-12385",  
      "action": "position",  
      "target": { "x": 10, "y": 0, "z": 5 }  
    }  
  ]  
}
```

7.1. Формат batch-сообщения

actions - массив команд

Каждый элемент массива должен содержать:

Id - идентификатор агента

action - тип действия

target - целевая позиция, если требуется

8. Поддерживаемые действия

8.1. position

Команда перемещения агента к точке.

```
{  
  "Id": "-12385",
```

```
"action": "position",  
"target": { "x": 10, "y": 0, "z": 5 }  
}
```

Назначение:

установить цель движения агента;

используется для перемещения к сокровищу, базе или любой другой точке.

8.2. pickup

Подобрать сокровище.

```
{  
  "Id": "-12385",  
  "action": "pickup",  
  "target": { "x": 10, "y": 0, "z": 5 }  
}
```

target может передаваться как вспомогательное поле, хотя в логике сервера основной смысл команды - сам факт взаимодействия.

8.3. drop

Сбросить сокровище.

```
{  
  "Id": "-12385",  
  "action": "drop"  
}
```

8.4. steal

Украсть сокровище.

```
{  
  "Id": "-12385",  
  "action": "steal"  
}
```

8.5. ready

Готовность игрока в лобби.

```
{  
  "Id": "-12385",  
  "action": "ready"  
}
```

Важно: в текущей серверной логике команда ready обрабатывается только пока игра не началась.

9. Правила обработки команд на сервере

9.1. Команды в лобби

Если состояние игры не InGame, сервер игнорирует все действия, кроме ready и для ready вызывает отметку готовности игрока в лобби.

9.2. Команды во время игры

Если игра в состоянии InGame, сервер ищет агента по полю Id, потом проверяет, что агент принадлежит команде клиента и выполняет действие.

9.3. Ограничение по команде

Клиент может управлять только агентами своей команды.

Если team агента не совпадает с командой сессии, команда игнорируется.

10. События игры

Сервер отправляет отдельные сообщения-события.

10.1. Старт игры

```
{
  "type": "gameEvent",
  "eventType": "start"
}
```

10.2. Завершение игры

```
{
  "type": "gameEvent",
  "eventType": "end"
}
```

10.3. Результат матча

```
{
  "type": "gameEvent",
  "eventType": "result",
  "winner": "blue",
  "scores": {
    "blue": 12,
    "red": 9
  }
}
```

Поля:

winner - победившая команда (blue, red или draw)

scores - итоговые очки команд

10.4. Сброс игры

```
{
  "type": "gameEvent",
  "eventType": "reset"
}
```

```
}

```

11. Примеры полного обмена

11.1. Подключение и вход в команду

Клиент → сервер

```
{"team":"blue"}
```

Сервер → клиент

```
{"type":"joinAccepted","playerId":"player_1","team":"blue"}
```

11.2. Отправка готовности

Клиент → сервер

```
{"actions":[{"action":"ready"}]}
```

11.3. Управление агентом

Сервер → клиент

```
{
  "tick": 102,
  "gameTime": 18.2,
  "gameState": "InGame",
  "bases": [
    {"team":"blue","pos":{"x":0,"y":0,"z":0},"points":3},
    {"team":"red","pos":{"x":20,"y":0,"z":0},"points":5}
  ],
  "agents": [
    {"agentId":"-12385","team":"blue","pos":{"x":5,"y":0,"z":4},"hasTreasure":false,"isStunned":false}
  ],
  "treasures": [
    {"id":"4521","pos":{"x":8,"y":0,"z":8},"isPicked":false}
  ],
  "golems": []
}
Клиент → сервер
{
  "actions": [
    {
      "Id": "-12385",
      "action": "position",
      "target": { "x": 8, "y": 0, "z": 8 }
    }
  ]
}
```

12. Логика клиента-бота по примеру

Пример клиента в Python показывает типичную стратегию:

1. Подключиться к серверу.
2. Выбрать команду.
3. Дождаться joinAccepted.

4. Отправить ready.

В InGame:

1. искать своих агентов;
2. находить ближайшее доступное сокровище;
3. идти к нему;
4. подбирать сокровище;
5. после подбора идти к своей базе;
6. у базы сбрасывать добычу.

13. Поведение при отключении

Если клиент отключается, то сервер удаляет его сессию, освобождает команду, закрывает сетевой поток и сокет и удаляет клиента из списка активных подключений.

14. Ограничения и особенности реализации

14.1. Одно подключение на команду

Сервер хранит список занятых команд. Пока команда занята, второй клиент с той же командой получит отказ.

14.2. Поток обновления

Состояние мира отправляется периодически, а не по событию изменения. Клиент должен быть готов получать множество повторяющихся snapshot-сообщений.

14.3. Формат сообщений

Сообщения должны быть строго JSON-совместимыми и завершаться \n.

14.4. Идентификатор агента

Поле `Id` в команде - это именно идентификатор агента, а не идентификатор игрока.

15. Рекомендуемый контракт для клиента

15.1. Подключение

Отправить `{ "team": "blue" }`

Дождаться `joinAccepted`

Отправить `ready`

15.2. В бою

Использовать только агентов своей команды

Не отправлять команды чаще, чем позволяет локальная логика управления

Отправлять batch-команды в одном сообщении

16. Минимальный набор сообщений

Клиент → сервер

```
{ "team": "blue" }
{ "actions": [ { "action": "ready" } ] }
{ "actions": [ { "Id": "-12385", "action": "position", "target": { "x": 10, "y": 0, "z": 5 } } ] }
```

Сервер → клиент

```
{ "type": "joinAccepted", "playerId": "player_1", "team": "blue" }
{ "type": "gameEvent", "eventType": "start" }
{ "tick": 1, "gameTime": 0.1, "gameState": "Lobby", "bases": [], "agents": [], "treasures": [], "golems": [] }
```

17. Краткая спецификация

Сообщения клиента:

Выбор команды

```
{ "team": "blue" }
```

Batch-команды

```
{ "actions": [ ... ] }
```

Сообщения сервера:

Подтверждение входа

```
{ "type": "joinAccepted", "playerId": "player_1", "team": "blue" }
```

Отказ во входе

```
{ "type": "joinRejected", "reason": "occupied" }
```

Событие игры

```
{ "type": "gameEvent", "eventType": "start" }
```

Состояние мира

```
{ "tick": 1, "gameTime": 0.1, "gameState": "InGame", "bases": [], "agents": [], "treasures": [], "golems": [] }
```

18. Примечания для интеграции

Клиента можно реализовать где угодно, достаточно поддерживать TCP и JSON.

Чтение сообщений должно быть построчным, до \n.

Нужно корректно обрабатывать частичные TCP-пакеты и склеенные сообщения.

Перед отправкой команд полезно хранить локальный кэш последнего состояния мира.

18. Пример псевдокода клиента

```
connect TCP 127.0.0.1:8080
send {"team":"blue"}
wait until joinAccepted
send {"actions":[{"action":"ready"}]}
loop:
  read world snapshot
  for each friendly agent:
    decide action
  send {"actions":[...]} when needed
```